# Improved Similarity Trees and their Application to Visual Data Classification

Jose Gustavo S. Paiva, Laura Florian-Cruz, Helio Pedrini, Guilherme P. Telles, and Rosane Minghim

**Abstract**—An alternative form to multidimensional projections for the visual analysis of data represented in multidimensional spaces is the deployment of similarity trees, such as Neighbor Joining trees. They organize data objects on the visual plane emphasizing their levels of similarity with high capability of detecting and separating groups and subgroups of objects. Besides this similarity-based hierarchical data organization, some of their advantages include the ability to decrease point clutter; high precision; and a consistent view of the data set during focusing, offering a very intuitive way to view the general structure of the data set as well as to drill down to groups and subgroups of interest. Disadvantages of similarity trees based on neighbor joining strategies include their computational cost and the presence of virtual nodes that utilize too much of the visual space. This paper presents a highly improved version of the similarity tree technique. The improvements in the technique are given by two procedures. The first is a strategy that replaces virtual nodes by promoting real leaf nodes to their place, saving large portions of space in the display and maintaining the expressiveness and precision of the technique. The second improvement is an implementation that significantly accelerates the algorithm, impacting its use for larger data sets. We also illustrate the applicability of the technique in visual data mining, showing its advantages to support visual classification of data sets, with special attention to the case of image classification. We demonstrate the capabilities of the tree for analysis and iterative manipulation and employ those capabilities to support evolving to a satisfactory data organization and classification.

**Index Terms**—Similarity Trees, Multidimensional Projections, Image Classification.

## 1 Introduction

Trees in visualization have been frequently used as a means to express multidimensional hierarchical data in various ways. Many visualization systems allow effective ways of exploring trees via various versions of link-node representations [32]. Widely used techniques such as Treemaps [2] improve space usage against conventional link-node representation and optimize the number of data set items that can be presented in a rectangular space.

In most cases, tree representations have been used in situations where: (i) hierarchy is the natural organization of the data and the tree reflects that; or (ii) data display is subject to a type of ordering of attributes and exploration is therefore attribute-based. In either case, trees are useful to explore data bounded by the hierarchy, and interpretation relies on a small number of attributes (those that can be mapped to visual artifacts such as color, texture and size).

Shortcomings of attribute centered visualizations start to appear when the number of dimensions grow. In this case, point-based strategies, defined by techniques where each individual in the display is an individual or group of individuals in the data set, offer a solid first step to explore the data. It relies on a relationship among individuals calculated using all the available attributes. In such cases, some type of dimensional reduction is commonly employed. Visualizations can be obtained by providing mappings of original or reduced spaces to visual (2D or 3D) spaces. Multidimensional projections are used to generate these mappings and the field has enjoyed great attention lately, resulting in more precise and faster approaches for point placement.

Similarity trees have unique properties concerning their ability to solve similarity-based (therefore, content-based) point placement. Additionally, they are an intuitive way of expressing structure in the relationship among items in the data set by adding hierarchy to the concept of similarity, that is, by seamlessly representing levels of similarity reflected as depth in the tree.

Regardless of their attractiveness, very little research has been carried out on similarity trees as a form of point placement for multidimensional visualization. The only technique reported in that context is the NJ tree (neighbor joining tree) [10], that has been shown to be a high precision tree structure in terms of reflecting on the display the neighborhoods found in the multidimensional space. Tests were made mainly for textual document collections, but its application for visualization of collections of images has also been reported [15], demonstrating qualities of NJ trees for image analysis tasks, such as visual support to feature extraction, selection, and comparison. In the latter work, interaction and coordination capabilities of trees and projections become evident, but no numerical evaluation is provided.

Another advantage of NJ similarity trees is to present, within a particular branch, the same properties of the global display due to their precise phylogeny reconstruction algorithm, largely explored in Biology [20]. Additionally, compared to other point placement strategies, point clutter is reduced with NJ trees due to the simplified graph layout algorithm that is used for display and due to the fact that branches allow direct inspection. Thus, there is no need for large gaps on the display window and all the space available can be used to plot points and edges. Their disadvantages are related to computational cost and the massive presence of virtual, non-representative nodes that use a substantial part of the visual space, reducing presentation area. The processing time constraint, though, is probably the reason why the technique has not yet reached wider usage.

In this paper, we propose a highly improved version of the NJ trees, both in space usage and processing speed, and illustrate their usefulness by developing an application that shows the adaptability of the approach for visual data classification tasks, with emphasis on image classification. A large number of applications can make use of this strategy. Classification is a process very difficult to converge and to adapt to a particular environment, especially when dealing with image data. In many cases, even when automatic classification algorithms give satisfactory results, visual feedback is very important to offer insight into the reasons for classification failure, and to support interfering with further classification passes, as well as intently correcting

- *J.G.S. Paiva is with Federal University of Uberlândia and ICMC, jgustavo@icmc.usp.br.*
- *L. Florian-Cruz is with ICMC - University of São Paulo, laurelyf@icmc.usp.br.*
- *H. Pedrini is with IC - University of Campinas, helio@ic.unicamp.br.*
- *G.P. Telles is with IC - University of Campinas, gpt@ic.unicamp.br.*
- *R. Minghim is with ICMC - University of São Paulo, rminghim@icmc.usp.br.*

*Manuscript received 31 March 2011; accepted 1 August 2011; posted online 23 October 2011; mailed on 14 October 2011.*
*For information on obtaining reprints of this article, please send email to: tvcg@computer.org.*

the classification. A proper visual set up is at the core of this problem. Some of the applications include product categorization, scene identification, photographs and music organization; and forensics.

The main contributions of this paper are: (i) the definition of a leaf promotion procedure that improves visual quality and space usage of NJ trees; the resulting technique is named Promoting NJ tree (PNJ tree); (ii) the adaptation and implementation of two faster versions of the neighbor joining strategy, reducing its computational cost and rendering the technique applicable to much larger data sets; (iii) objective numerical evaluation of the quality of the final displays; and (iv) the illustration of an application where similarity visualizations displayed by trees are employed to help the user in various steps of the data classification process. We focus on image classification and also exemplify using text.

The remainder of this paper is organized as follows. Section 2 reviews the literature on content based point placement of multidimensional data. Section 3 describes similarity trees, explains the faster algorithms and introduces the leaf promotion procedure. Section 4 presents analysis and comparative results of the experiments with similarity trees, while Section 5 presents the tree-based tools for the visual classification application. Section 6 concludes the paper.

## 2 Related Work

Point placement strategies aim at mapping each individual data point into a visual space. While basic mapping is most commonly based on dissimilarity calculations, some of their other attributes can also be mapped as geometrical shapes, colors, textures and even sounds. Relationships can also be explicitly represented in the visual space as edges between points.

Many of the most demanding current applications produce data sets with a large number of data items, represented by a large number of attributes, or dimensions, imposing serious constraints to information visualization and analysis techniques.

A recurring way to approach high multiplicity of dimensions or attributes is to use dimension reduction strategies, by selecting, mapping or combining dimensions in order to apply conventional visualization methods that are prepared to handle low dimensional data. Layout approaches can also be based on multidimensional projections, which map a high dimensional space into a visual space trying to preserve, in the final space, a relevant relationship among data elements in the original space.

Most 2D or 3D multidimensional projections employ a measure of dissimilarity between data elements. Thus, data elements mapped as points to the same region in visual space are considered similar. Interpretation of the layout is accomplished by locating groups of interest and focusing on such groups and their subgroups.

Figure 1 shows a layout of 675 extracts from scientific papers built using the Least Squares Projection [30]. On this type of projection, proximity between points in the layout is meant to indicate document similarity. An automatic topic detection procedure yields a glimpse of the subjects handled by a document group.

Several multidimensional projection techniques are available. Most of them are based on dimension reduction techniques and some of those have been largely used for data analysis for decades. Principal Component Analysis (PCA) [23] is a widely used linear projection technique that employs linear combinations of the data attributes (dimensions) with a high covariance degree, producing attributes with less dependence. Its main disadvantage is the low quality of resulting layouts due to the relatively low power of the two or three principal dimensions to express important features in heterogeneous data sets. Its computational cost is also high, $O(nm^2)$, where $n$ is the number of objects and $m$ is the number of dimensions. Multidimensional Scaling (MDS) [9] comprises a class of techniques that can be used to perform projections. The simplest MDS approach is the Force Directed Placement (FDP) model [13]. The FDP model is based on a spring system concept, where the multidimensional instances are modeled as objects linked by springs such that the repulsion and attraction forces between the objects are proportional to the multidimensional distances. The projection is attained when the spring system reaches an equilibrium state. This technique normally presents high precision for data sets with objects possessing non-linear relationships, but has high computational cost, $O(n^3)$ and uneven precision across different types of data sets (such as documents and images). The Force Scheme [38] and the strategy defined by Chalmers [4] are also based on the concept of attraction and repulsion forces among objects. Both use heuristics that decrease FDP's running time to $O(n^2)$. As a faster, high precision projection, the Least Square Projection (LSP) [30] applies a Laplacian operator in which neighbor data in the multidimensional space are projected to close positions on the visual layout. LSP presents a proper balance between precision and running times, being an $O(n\sqrt{n})$ technique.

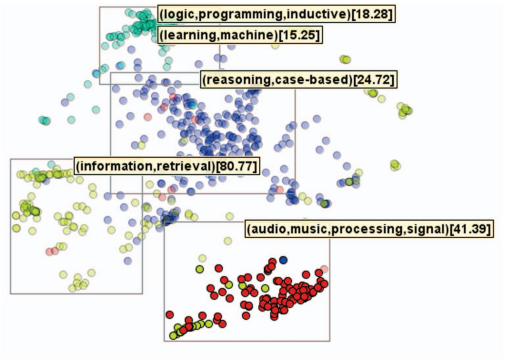



Fig. 1. Projection of 675 extracts (title, authors, abstract and references) of scientific papers. Circles represent documents, colored according to subjects: case-based reasoning (blue), inductive logic programming (green), information retrieval (yellow), and computer audio (red). The audio group is highlighted.

The previous methods generate good results for multidimensional data with varying precisions. In particular, it has been shown that LSP consistently approximates highly related data points and separates groups well. Layouts resulting from LSP provide a powerful guide to recognize global patterns in multidimensional data and help user to build a mental model of the data set [30].

Many new dimensionality reduction techniques for multidimensional projections have been put forward lately [5, 11, 28, 31] with motivating results. With this evolution, their number of applications have increased, particularly in recent years. They are capable of handling large data sets and a large number of dimensions. They have also been in the frontier of integration between spatial and non-spatial visualization techniques.

Regardless of their advantages, projections are not usually capable of making locally the same claims of precision made globally. Within a certain group of points, there is significantly less neighborhood consistency that, in principle, the similarity relationship should be capable of coding. Hierarchical approaches can be a solution to that particular problem [29], but they rely on early clustering, preventing proper analysis of boundaries between groups. Newer and extremely fast projections have been put forward lately, handling well the problem of computational cost with improved precision [31] or the problem of consistency in local neighborhoods [28]. However, they work with space partitioning and cannot handle similarity matrices, impairing their use with applications that define similarity relationships without undergoing feature space definition (for instance, textual similarity from string comparison or similarity based on other non-numerical data). Projections are also subject to a high degree of cluttering due to the algorithm effort in guaranteeing the neighborhood between highly correlated neighborhoods. Visual density estimation, in that context, is virtually impossible without further interaction.

Multidimensional projections are evaluated by their capability of grouping and separating strongly related data items and by their capability of recovering, in visual spaces, data distribution or neighborhoods in original space. They have been proven to excel in both. Similarity trees should be evaluated in the same manner.

As an alternative technique for exploring multidimensional data based on content, the data set can be organized as a similarity tree. By positioning objects on branches, similarity is organized into levels, a natural way of interpreting degrees of similarity. Global analysis is not impaired, that is, it is also possible to distinguish larger groups of interest within the data set. Local analysis is as precise as global, sometimes more precise. Similarity tress can be built from feature spaces or similarity relationships. This is the case of the NJ tree [10], a visualization technique based on a phylogeny reconstruction algorithm named Neighbor Joining. This paper addresses this alternative, improving Cuadros et al.'s technique in both visual and processing time scales and validating its use for visual data classification.

## 3 Similarity Trees for Multidimensional Visualization

This section describes NJ trees and proposes alternatives for improvement of both time and visual scalability. The result is a faster and less cluttered hierarchical visualization based on similarity, that completes the typical functionality of a multidimensional projection.

In terms of tree construction, the technique encompasses a two step procedure: after the construction of the similarity data structure representing a tree, the final layout is produced by a tree drawing algorithm. The input for tree construction algorithms is a symmetrical similarity matrix, and finding a similarity measure that faithfully represents relationships among objects in high dimensional feature space is a key issue for both trees and projections.

Addressing the quality of the similarity relationship is beyond the scope of this work. We consider that proper feature extraction and similarity calculation precede the visualization step and that the tree is responsible to reflect that feature space and similarity, although the tree itself could be a valuable tool for feature space convergence procedures.

Next, we explain and exemplify NJ trees, reporting on their known features. We also show their limitations as well as our solutions to improve those limitations. Additionally, an evaluation methodology for similarity trees is introduced.

### 3.1 Neighbor Joining

A phylogenetic tree represents evolutionary relationships within a group of species. Leaves represent actual species and internal nodes represent hypothetical ancestors. Edges represent ancestor-descendant relation and their lengths may indicate distance between species. A phylogenetic tree may be rooted or unrooted [35].

In the work by Cuadros et al. [10], unrooted phylogenetic trees were built for documents and served as visualization models. Tree leaves represent documents, internal nodes represent hypothetical documents with intermediate content, and edge lengths indicate the similarity among documents. Because phylogenetic tree construction is NP-hard in general, the authors used a well-known heuristic that builds unrooted trees from distance matrices named *Neighbor Joining* (NJ) [35]. The trees are named NJ trees. Figure 2 shows an example of an NJ tree for the collection of documents of Figure 1. It shows that group organization in branches is consistent with the LSP projection, but it also structures the groups into layers of similarity and reduces cluttering considerably. Local neighborhood is also reconstructed in NJ trees according to the similarity matrix.

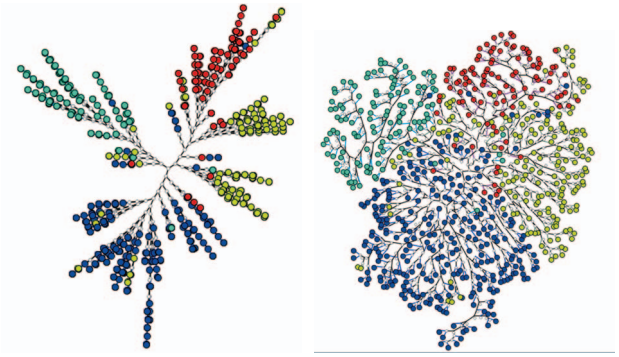
We refer to the previous work [37] for implementation details of Algorithm 1. The input is a similarity matrix $D$ and the output is a phylogenetic tree with $n$ leaves and $n-2$ internal nodes with degree 3. Its running time is $O(n^3)$. This is clearly a limiting factor for very large data sets. In Algorithm 1, $R_i$ is the average distance from node $i$ to every other node in $D$. $S_{ij}$ captures the notion of evolutionary change, and, at each step, two closest nodes in $D$ are removed from the matrix and replaced by a combination (virtual) node $x$, for which a new row is inserted in $D$. The new distances from $x$ to every other remaining node are calculated according to the formulae in Algorithm 1.

NJ trees were successful in building visualizations that consistently separate texts with similar content in different branches. It also favors data exploration in various levels of detail. The technique can also be applied to other types of data, such as image and sound [10].

**Algorithm 1** Neighbor Joining

1: Let every row $i$ of matrix $D$ be associated with a leaf $i$.
2: **repeat**
3: Select a pair of nodes $(i, j)$ with the minimum value for $S_{ij}$
4: Create a new node $x$ connected to nodes $i$ and $j$, with edge lengths $L_{ix}$ and $L_{jx}$
5: Add row $x$ to $D$, with values $D_{xk}$ for every column $k \neq i, j$
6: Remove rows $i$ and $j$ from $D$
7: **until** $n = 3$
8: Connect the three remaining nodes in the tree

$$S_{ij} = D_{ij} - R_i - R_j;\ R_i = \frac{1}{n-2}\sum_k D_{ik}$$
$$L_{ix} = \frac{1}{2}(D_{ij} + R_i - R_j);\ L_{jx} = D_{ij} - D_{ix};\ D_{xk} = \frac{1}{2}(D_{ik} + D_{jk} - D_{ij})$$



(a) Initial layout. (b) Layout after fast force scheme.

Fig. 2. NJ tree for a collection of 675 textual documents.

After NJ tree construction, a linear radial graph layout algorithm [1] is applied, resulting in the nodes separated in larger branches (Figure 2a). Then, applying a fast simplified force-based layout [38] nodes are spread minimizing cluttering and allowing inspection of branch organization (Figure 2b).

Regardless of the motivating visual exploration capabilities and aesthetically pleasing layout, in a collection with $n$ objects, $n-2$ internal nodes will be created. The space is overloaded by these virtual nodes that neither represent information nor add to the concept of multi-level similarity.

Figure 3 shows an NJ tree with its virtual objects highlighted, showing that the visualization model is visually polluted even for such a moderate sized collection. For larger collections, the problem becomes more serious.

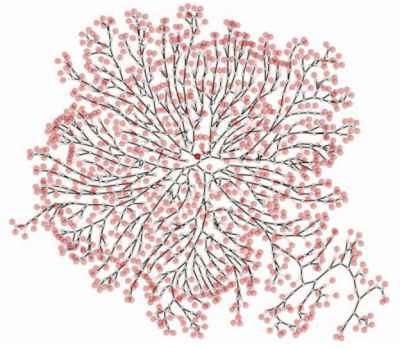


Fig. 3. NJ tree for 890 items with its 888 virtual objects highlighted.

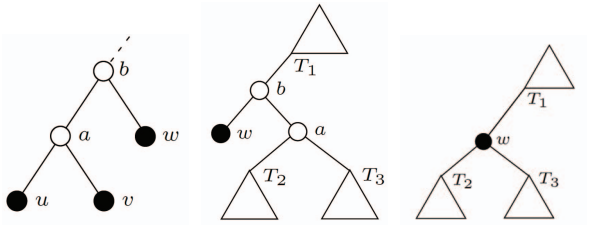
The procedure described next intends to improve the layout regarding its visual space usage.

### 3.2 Leaf Promotion in NJ trees

In order to reduce visual overload in NJ trees, we present a deterministic, ordered graph rewriting operation [14, 34] that replaces an internal node with a leaf whenever a certain configuration of nodes exists. This operation is called promotion.

Suppose a NJ tree $T$ such that there exists a pair of leaves $u$ and $v$, both connected to the same internal node $a$, and that another internal node $b$ connects $a$ and a leaf $w$, as shown in Figure 4(a). The rationale behind promotion is that because no other node was closer to $w$ than $a$ (during the construction of $T$), then no other node is closer to $u$ and $v$ than $w$, so $a$ can be replaced by $w$ and $b$ can be removed altogether, with no loss of representational power. This relation among $u$, $v$, $w$ and $a$ is weak, because it is valid only for the topology created by the NJ algorithm and it does not hold for the metric space induced by the similarity matrix (if any exists). Nevertheless, experiments have shown that the layout precision is barely affected by promotion.

The operation may be formally defined in terms of a pattern and a replacement on the tree, as shown in Figures 4(b) and 4(c). The promotion procedure consists of replacing every occurrence of the pattern, in decreasing order of the distance from node $a$ in the pattern to a node in the center of the tree, breaking ties arbitrarily. Weights in the replacement are denoted $\omega_r$ and are evaluated from the weights in the pattern, that are denoted $\omega_p$ and were computed by the tree construction algorithm. We abuse notation and use $T_1$, $T_2$ and $T_3$ to denote the node connecting a subtree to the rest of the tree. We let $\omega_r(w,T_1) = \omega_p(b,T_1) + \omega_p(a,b)/2 + \omega_p(w,b)/2$ and $\omega_r(w,T_i) = \omega_p(a,T_i) + \omega_p(a,b)/2 + \omega_p(w,b)/4$ for $i = 2,3$. The resulting tree will be referred to as PNJ tree. Linear time suffices to find the nodes that match the pattern, the center of the tree, and to evaluate the distances. Sorting takes $O(n \log n)$. Applying the pattern takes constant time, then the overall promotion takes $O(n \log n)$. Experiments have shown that computational time to perform promotion is negligible.




(a) example (b) pattern (c) replacement

Fig. 4. Promotion. Filled circles represent actual objects and triangles represent subtrees.

### 3.3 Faster Algorithms for Neighbor Joining

In the original NJ, the search for the minimum $S_{ij}$ in a matrix $D$ (which determines the next pair of nodes to combine) essentially visits every cell in $D$ and thus takes $O(n^2)$. This is the target operation of NJ variations that aim at improving efficiency, either by using specialized data structures [25, 36, 40] or through heuristics that sacrifice precision [16, 17]. We have investigated two of them to assess the scalability of NJ trees. Our descriptions below are based on the original NJ steps and on the notation presented in Algorithm 1. Additionally, let $S_{min}$ denote the minimum $S_{ij}$ for a matrix $D$.

Rapid Neighbor Joining [36] produces the same trees as NJ. It is based on the observation that, while visiting row $i$ and evaluating $S_{ij} = D_{ij} - R_i - R_j$ in the search for $S_{min}$, the value of $R_i$ is fixed. The algorithm keeps an auxiliary matrix where rows are sorted (and whose cells are indexed back to $D$) and stops visiting the sorted row $i$ as soon as $S_{ij} - R_i - R_{max} > S_{min}$, where $R_{max}$ is the maximum $R_i$ among all matrix rows (thus $R_j \leq R_{max}$.) Some overhead is added by sorting rows, handling removed columns and evaluating $R_{max}$. While Rapid NJ is still $O(n^3)$, experiments have shown that the bounding strategy significantly reduces the number of visited cells in the search for $S_{min}$ and that Rapid NJ outperforms other faster algorithms.

A different approach is adopted by Fast Neighbor Joining [16]. Fast NJ maintains a set having $O(n)$ candidate matrix cells. The search of $S_{min}$ is restricted to such set of candidates. Initially, the set of candidates contains the minimum $S_{ij}$ values for each row $i$. When rows $i$ and $j$ are connected to a new node $x$ and removed from $D$, the candidates they contribute are removed and a new candidate for the new matrix row $x$ is added. The approach results in an algorithm that does not produce the same tree as the original NJ and whose worst case is $O(n^2)$. The authors have shown that when the distance measure is nearly additive, Fast NJ produces the same tree as NJ. We devised and adopted a more conservative variation of Fast NJ. Exactly $n$ candidates are kept, one for each row. When rows $i$ and $j$ are connected to a new node $x$ and removed from $D$, we remove the candidates they contribute and we also recompute remaining candidates that involved either $i$ or $j$. Recomputing the candidates potentially improves the choices of $S_{min}$. The modified algorithm is $O(n^3)$, but the practical running times are much closer to $O(n^2)$.

We further discuss running time and precision issues in the next section.

## 4 Results

We have added the new strategies and algorithms to a visualization platform called VisPipeline. VisPipeline implements various multidimensional visualization techniques, mainly focused on content-based point placement techniques, supported by interaction, preprocessing and some analysis tools. VisPipeline is implemented in Java and allows components and tools to be connected visually. We then specialized some of the tools and created the visual image mining framework presented in Section 5. The two faster versions of the NJ tree, along with their versions with node promotion, were adapted and implemented. The available trees are: the original version (NJ), the acceleration implemented by the Rapid Algorithm (Rapid NJ), the Fast NJ modified as previously stated (FAST), and the promoting strategies for each one of these, which we will refer to as PNJ, PRapid and PFAST.

We have added numerous interactions as well as a few mining and evaluation functions, to adapt currently point placement analysis to the similarity tree concept. Some of the added features include: tree based interactions for cutting, selecting, splitting, browsing and coloring; numerical evaluations for the tree layouts; visual analysis procedures, such as clustering and classification algorithms (thus far SVM [7, 22], KNN [8, 12] as well as various hierarchical clusterings are implemented); and similarity perturbation procedures to perform class visualization from an initial visual layout of labeled instances.

For the tests of processing times, we have employed a laptop with a 2.53 GHz Core2 Duo processor, 4GB RAM, running Windows 7.

This section presents the results of applying and testing the new versions of the similarity trees for improvement on visual and time scalability.

We firstly evaluate the precision of the original NJ tree against a high precision projection, LSP. The first paper that proposes the use of phylogenetic trees for visualization [10] also does that, but it employs various textual document data sets, and we wished to test it against imaging data sets, due to our focus application. Although a previous work uses the tree for visualization of images [15], these evaluations were not previously performed. Secondly, we evaluate the precision of the trees against one another. Finally, we compare the processing times of all these techniques and demonstrate their capability to support exploration both in overall and detailed levels. First, we describe the data sets employed in the tests.

### 4.1 Data Sets and Test Setup

In the demonstrations and evaluations that follow, we have chosen to employ three imaging data sets, since this is the source of the sample application in Section 5.

The first image collection, named COREL, is composed of photographs that represent specific subjects: African tribes, beach, build-

ings, buses, dinosaurs, elephants, flowers, horses, mountains and glaciers, and food. Each image is represented by a vector of 150 SIFT descriptors [24]. The second image collection, named MEDICAL, is composed of MRI medical images, each represented by a vector of 28 descriptors, including Fourier descriptors from image histogram and energy computed from 2D image Fourier descriptors, mean intensity, and standard deviation computed over the entire image. The third collection, named OBJECTS, is a subset of the Caltech 256 data set, containing 45 classes, 5096 images and 1000 attributes that are 1000 SIFT features calculated over Harris-Laplace extractors [21]. For all data sets, the similarity used to compare the samples was Euclidean similarity. All data sets are labeled and these labels are used for the purpose of evaluation of the visualization and mining procedures reported here. Table 1 summarizes the numbers related to these data sets.

Table 1. Collection details.

| Data set | Content | Classes | Items | Attributes |
|---|---|---|---|---|
| COREL | Photographs | 10 | 1000 | 150 |
| MEDICAL | MRI images | 12 | 548 | 28 |
| OBJECTS | Object pictures | 45 | 5096 | 1000 |

The first aspect of the trees to be evaluated is the use of visual space.

### 4.2 Visual Representation and Space Organization

In the NJ tree, most of the larger groups of elements lie on separate branches, forming subtrees of the main tree. As a consequence of the algorithm, the branches are also divided into smaller branches with the same properties. Exploration and interpretation can be made equally in all levels. Selection of the branches, and therefore of groups of collection elements, can be made with one click of the mouse. On the other hand, interpretation is dependent on branches, that is, users need the edges representing the branches to recognize elements with large correlation. Opposite to multidimensional projections, visual space must be shared with the edges.

The main drawback of NJ trees regarding space usage, though, is the visual overload imposed by the presence of virtual objects (white small circles on the layouts), created every time two leaves are added to the tree or a node is combined with an already present virtual node. For a data set with $n$ elements, $n-2$ virtual nodes are added by any of the algorithms. As these objects are not part of the collection, they do not play any important role during analysis.

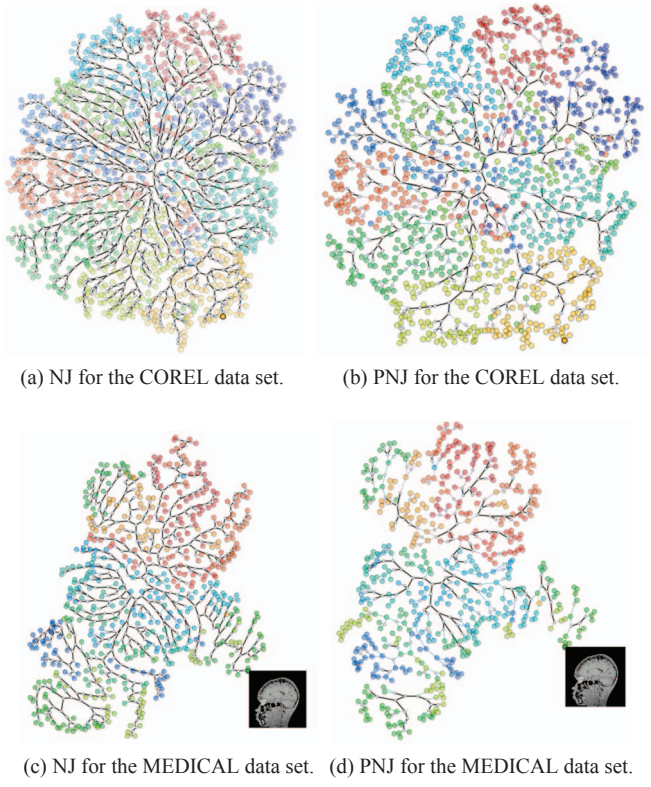
Node promoting shares the visual organization and exploratory advantages of NJ trees, but with fewer virtual objects, resulting in a cleaner view and ultimately leading to a more direct access to actual data objects on the tree branches. In our tests, PNJ presents in average 51% fewer virtual nodes than NJ trees. Table 2 summarizes the numbers related to space usage on the visualizations of the test data sets.

Table 2. Numbers of Nodes.

| Data set | Nodes | Virtual | PNJ virtual | Saving | PFAST virtual | Saving |
|---|---|---|---|---|---|---|
| COREL | 1000 | 998 | 412 | 59% | 548 | 45% |
| MEDICAL | 540 | 538 | 186 | 65% | 308 | 43% |
| OBJECTS | 5097 | 5095 | 2437 | 53% | 3019 | 41% |

For PNJ, space usage is much more rational and clutter is reduced. Figure 5 shows the images of two of the test data sets, where we can clearly see the impact of having fewer virtual nodes. For larger data sets, the impact is even more evident.

It must be observed that the promotion procedure disturbs the 2-neighborhood of the nodes. That changes slightly the distance distribution (see Section 4.3), but should not impact interpretation of the trees or other neighborhoods significantly. Large layers of virtual nodes at top levels, typical of NJ, are avoided. The Rapid algorithm for



(a) NJ for the COREL data set. (b) PNJ for the COREL data set.

(c) NJ for the MEDICAL data set. (d) PNJ for the MEDICAL data set.

Fig. 5. Tree models for the COREL and MEDICAL data sets, highlighting virtual nodes and paths.

NJ leads to the same configuration as the original NJ. Thus, PNJ and PRapid are the same as far as virtual node count. Trees produced by the FAST algorithm have the same node count as NJ, although producing a different node configuration. FAST node configuration leads to a lower reduction by promotion, as Table 2 reveals. With node promotion, space can be occupied by further nodes or used to easier access to groups of individuals.

### 4.3 Precision

Various multidimensional projections have high precision in terms of grouping and separation, as well as very competitive processing times for large data sets. Similarity trees also have high precision but algorithm complexity of previously available versions and consequent processing times were far behind those of the high precision projections, making the technique less appealing for large data sets, regardless of its qualities. Here we show the evidence of precision and compare the precision of all the algorithms as well as with the precision of LSP.

Measuring the precision of a particular layout is meant to verify to what degree the distance (or neighborhood) in the layout corresponds to the similarity (or neighborhood) in the original feature space. In the layout generated from multidimensional projections, the Euclidean distance is used to indicate dissimilarity between points according to that projection. In trees, the dissimilarity between points is indicated by the weight of the path between them. Edge weights in the tree are evaluated by the NJ algorithm ($L_{ix}$ and $L_{jx}$ in Algorithm 1).

To evaluate precision, two measures previously employed for projections [30] were adapted to the trees. The first measure is the *Neighborhood Preservation*, that evaluates the frequency at which the $k$-nearest neighbors of an object in the layout are also its $k$ nearest neighbors in the original feature space, in average. Since the Rapid algorithm produces the same trees as the original algorithm, precision discussions on Rapid are the same as the discussion on the original NJ.

For the COREL data set, the trees show better precision than that of LSP. Also, it becomes clear that near neighborhoods are reconstructed

better for the trees than for LSP (Figure 6). Among the trees themselves, it can also be noticed that the ones created by FAST present slightly lower precision than the corresponding ones generated by original NJ algorithm. Another observation is that the promoting version for each algorithm is slightly less precise in neighborhood preservation than the non-promoting ones. That is easily explained by the fact that the 2-neighborhood of a point changes when a node is promoted, since sometimes it becomes the parent of a couple of other nodes that were deemed the closest in the first place (see Section 3.2). In that case, the parent is interfering with the closeness and with the concept of first neighbor. That is also the reason for the low value of neighborhood preservation for the first neighbor on those trees. In larger neighborhoods, the relationships are not changed by promotion. Yet, the trees presented very good neighborhood preservation considering the difficulty in recovering that for any multidimensional data mapping. The neighborhood preservation plot among trees for the MEDICAL data set is very similar to the plot for COREL, but there is no significant difference between the precisions of trees and of LSP, except for the very near neighborhoods, where global projections such as LSP do not perform so well.

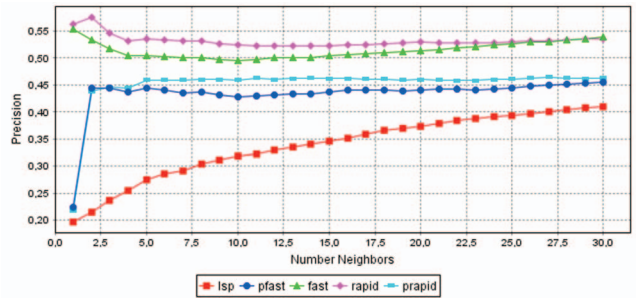



Fig. 6. Precision by Neighborhood Preservation for the COREL data set.

When a data set used for tests is labeled, that can also be used to verify precision based on class reconstruction in the final layout. We would like to point out that feature space definition and dissimilarity calculations play an important role in any visualization or mining process, and much so in the precision of similarity-based visualizations, particularly those based on pseudo-classes. For the COREL and MEDICAL data sets used here, the feature space was obtained or calculated, and various tests were carried out, so that the result described reasonably well the set of images, albeit without resolving class separation.

The second precision measure is called *Neighborhood Hit*. It is the average frequency at which an object and its $k$-nearest neighbors belong to the same class. Figure 7 shows the results of neighborhood hit for the COREL data set. Here, the trees also outperform LSP, but the capability of class reconstruction of the promoting trees does not differ much from that of the original algorithms. The performances for all the techniques were not significantly different. For the neighborhood hit plot based on the MEDICAL data set, the observations on the tree precisions also hold. However, there is no significant difference between the performance of LSP and that of the trees.

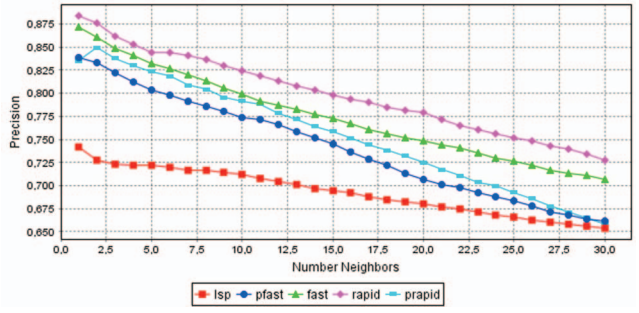



Fig. 7. Precision by neighborhood hit for the COREL data set.

Neighborhood hit is a value extremely influenced by proper preprocessing tasks (choice of features and similarity) and by the boundaries between groups. To help determine to which extent a 2D mapping (or 3D for that matter) is actually reflecting in the visual space the same distances as in the original space, a distance plot is commonly employed. A distance plot is a scatter plot of distances in original space versus distances in the visual space. Ideally, the point cloud of that plot will lie close to the $45^\circ$ slope.

Figure 8 shows distance plots (also known as stress curves) for the trees and for LSP. It can be seen that, for the COREL data set, the trees do a better job than the projection, with the original NJ algorithm performing best, followed closely by FAST. Promotion dissipates the distances around the diagonal, however, keeping the slope trend and the distribution very satisfactory. For the MEDICAL data set, LSP performs better at reconstituting the distances according to dissimilarity in original space, with the trees performing worse and very similarly among themselves. Dissipation under promotion is also observed, but to a lesser extent. Promotion over the FAST-NJ is not shown, but it had the same effect of slight point dispersion in both cases.

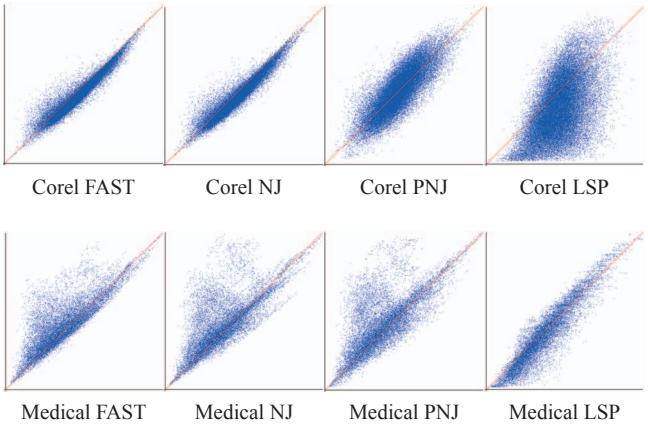



Fig. 8. Distance plots, with original space in horizontal axis vs. visualization space in vertical axis. COREL data set is top row, MEDICAL data set is bottom row. Techniques are FAST-NJ, NJ, PNJ, and LSP.

### 4.4 Efficiency and Scalability

Fast projections such as LSP, which takes time $O(n\sqrt{n})$, take advantage over the trees under time efficiency issues. However, the new versions of the NJ, based on FAST and Rapid algorithms, have dropped that difference. In fact, processing times for the FAST version are faster than the projection. The promotion procedure is efficient, so its cost is not significant.

Table 3 shows the time comparison between the various versions of the tree, as well as the projection, for the three test data sets. From that table, it can be seen that the trees have very competitive time consumption, with the FAST versions largely outperforming all the others.

Table 3. Layout Building Times (seconds).

| Technique | COREL | MEDICAL | OBJECTS |
|---|---|---|---|
| Original NJ | 3.0474 | 0.5 | 394.383 |
| Promoting Original NJ | 3.064 | 0.506 | 396.507 |
| Fast NJ | 0.0936 | 0.0462 | 4.976 |
| Promoting Fast NJ | 0.1104 | 0.052 | 6.1 |
| Rapid NJ | 2.366 | 0.373 | 261.971 |
| Promoting Rapid NJ | 2.3828 | 0.3788 | 262.178 |
| LSP | 0.7182 | 0.2068 | 61.564 |

Analyzing the processing times of Table 3 together with the precision discussions in the previous section, the scenario seems to point to a setup where global, initial layouts would be carried out using projections or the FAST or PFAST versions of the tree. More focused vi-

sualizations would take advantage of the better precision of the Rapid variation of NJ. Next, we illustrate the use of the tree as basis for a visual classification system.

## 5 Visual Image and Text Classification

Techniques have been proposed for employing point placement, particularly multidimensional projections and graphs, to display collections of images [6, 15, 26, 27, 33, 39, 42] with or without support for classification tasks, some of which already employ tree visualizations to reflect a hierarchical clustering [18, 19], serving the main purpose of supporting the browsing of collections on a visual space. However, to our knowledge, the tight coupling between a visualization system and an image mining environment is not available to date.

We focus on the goal of visually supporting image classification due to the challenges facing the subject, some of which we believe can be resolved by extensively involving the user via visual analysis tools. One of the challenges is the difficulty in understanding the reasons for failure of automatic classification procedures (e.g., what points are misclassified and why). Another recurrent situation is when users do have a proper automatic classification procedure in place, but wish to 'see' it and change it at will if necessary. The third scenario is when the current labeling of the data set is not appropriate for the goals of the user. There are different definitions of similar images and class labeling may acquire different configurations depending on the target applications. For both browsing and mining, we believe that similarity trees offer a valuable layout, for their capacity of local as well as global space reconstruction.

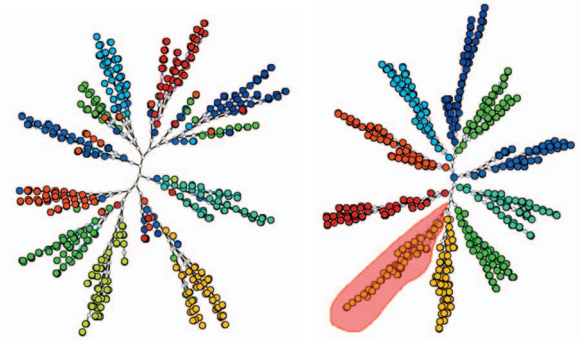
We have implemented a suite of tools for iterative classification, whereby the user, starting from either a labeled or a non-labeled data set, builds up training sets into classified sets by mixing automatic and visually supported manual classifications. The system, complete with evaluation tools, is added to a locally built data flow visualization prototype, named VisPipeline.

We describe some of the tools built around the concept of tree visualization to support tasks in such an integrated classification setup. The main features are presented in the next subsections and we finish by mentioning additional functionality implemented.

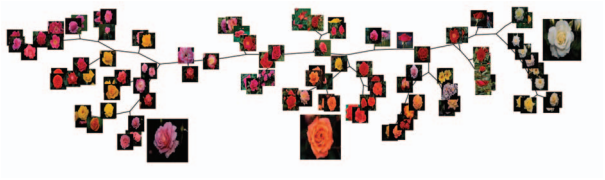
### 5.1 Layers of Similarity

Similarity trees allow the viewer to analyze various degrees of similarities and to explore the data in various levels of detail with practically the same strategy. Well resolved groups are laid out as branches, and the delimitation of such groups is given by the top layers of the unrooted tree, suggesting branch-based interaction. What is motivating about similarity trees is, therefore, that the model is capable of describing, to a certain extent, the organization of the similarity relationship defined or chosen by the analyst. If the dissimilarity measure is capable of distinguishing groups and subgroups, they should appear as branches in the tree.

Figure 9 shows an example of some of the drilling down capabilities of the trees. We first display the COREL data set using PNJ (Figure 9a). Then, since the data was already labeled, we perturbed the distance matrix by approaching the nodes that belonged to the same class and increasing the distance between nodes that did not belong to the same class. The result of perturbing the COREL distance matrix by 18% can be seen in Figure 9b. Although that type of perturbation can only be done when a correct label is established for the collection, it is very useful to separate classes in branches, while still maintaining, within each class, the structure and ordering of the original similarity. Figure 9c shows a branch containing flower pictures, and within that, evidence that color features may be separating the group further (that is, pink and most yellow flowers, orange and red ones, then white flowers). Selections can be made by circling groups of nodes in a free curve (Figure 9b) or by just clicking on any node. Clicking on a node selects either the node itself (if it is a leaf) or a branch. What the perturbation tool does, visually, is to redistribute those nodes that were close to the main skeleton of the tree towards their proper branch. Drilling can, of course, be done in any tree, but perturbation has interesting visualization value. A proper tool to generate a family of perturbed matrices according to users' specifications, as well as corresponding views, is made available. The various visualizations are displayed using a slider that moves from one perturbation to the next.



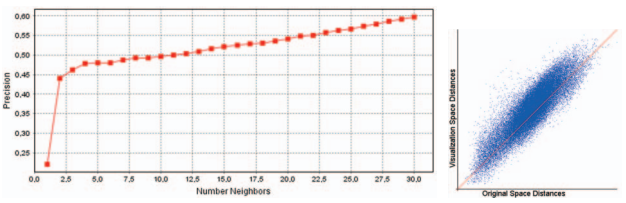
(a) PNJ of the Corel data set. (b) PNJ from a distance matrix perturbed by a rate of 18%.



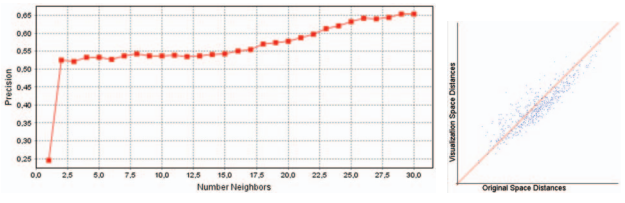
(c) Detail of the circled branch in (b), with flower pictures.

Fig. 9. Branch selection and exploration of the COREL data set.

To confirm the evidence that similarity trees keep, at lower levels, the same properties observed at the top level, we have plotted, for the visualization in Figure 9b and for each one of its branches, the neighborhood preservation and the distance plots. Figure 10 shows these plots for the full set and for the flower branch, but the results are consistent for all 8 branches. Figures 10d and 10c reveal that the local neighborhood and distance reconstitution follow the same patterns, or better, than the layout at global levels. All these evaluation functions, plus the ones used to evaluate the trees in Section 3, are readily available either as modules of the pipeline or on the menu and toolbox of the visualization window.




(a) Neighborhood preservation of COREL - 18%. (b) Distance plot of COREL - 18%.




(c) Precision of the circled branch in 9b. (d) Distance plot of circled branch in 9b.

Fig. 10. Precision consistency in branches.

The matrix perturbation process, besides supporting visualization of pre-defined classes, helps understand the layering of similarity and

supports our visual, interactive and iterative semi-automatic classification framework. When the matrix is perturbed, precision values improve even when the layouts are evaluated against the original matrices. One way to use that is to employ perturbation after the classification process, to help gather equally labeled data in the same branches and then browse branches to adjust wrongly classified ones. The process can also be useful to create new labeled data sets or to change pre-labeled ones to adapt to a new categorization scheme.

### 5.2 Manual Classification Tools

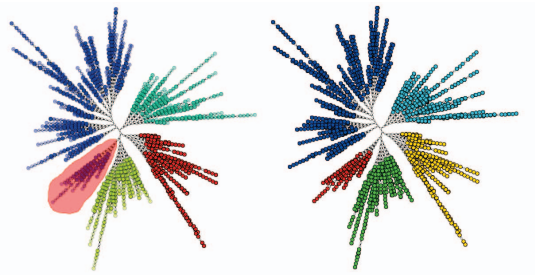
A visual method for classification has the advantage that the user is immediately exposed to the results of false positives, false negatives, mismatches and outliers. It also opens space for user's influence on the setup and on the outcome of the classification process. Users may want to relabel the training set in order to correct final classification or to assign labels themselves to unlabeled data sets. Relevance feedback techniques [3] could also use visualization tools. To support these tasks, we devised two manual classification tools, one to define an initial labeling for a non-labeled data set, and another to adjust categories of individuals or groups of individuals while browsing through the images. Both techniques are combined to perform a fast labeling or re-labeling task.

Figure 11 illustrates both tools. The OBJECTS data set was used in the example. Figure 11a shows the label-by-selection tool. Once a tree visualization is displayed for any data set, labeled or not, the user can define new labels (by color) for whole branches or for any group of nodes. Branches can be selected just by clicking at the top node of the branch, and other groups are selected using the free curve selector mentioned before. Each time this labeling procedure is started, a new scalar field is created with the new labels. When the user moves from one group to another, the labels are recolored for consistency. The latest selection in the example is colored red. The initial selection is blue. These are the extremes of the color table used in the examples. Once any color scheme is available for the data set, the user can browse images and change labels individually. Figure 11b illustrates this tool. After selecting a particular branch for browsing, users can change labels freely. New labels are defined by choosing new names for them or employing an already existing label name. The picture shows the selection of a few images on a branch. The name of the class is changed to "not round". Then, in the branch, whose visualization is seen in another window, the corresponding nodes change color. In Figure 11b, the change in color is highlighted by an ellipse. Once the data is fully labeled, these labels can be saved together with the original point or similarity matrix and used as training set for an automatic classifier or to evaluate other classifications of the same data.

### 5.3 Iterating Towards a Fully Classified Data Set

With the proper visualization tools, it is possible to envisage an iterative process to converge from a smaller labeled data set to a proper categorization of even very large data sets. A combination of automatic and user-driven tasks should compound an environment fit to help a considerable number of applications.

We have built another set of tools into VisPipeline to accomplish that employing the similarity trees as one of the main visualization techniques, since similarity is at the core of many classification tasks. Besides the tools and evaluation modules illustrated above, automatic classification algorithms are being progressively built into the system. A network of modules summarizing a classification setup is shown in Figure 12. One of the three pipelines in that picture is the visualization of the test set, another is the visualization of the training set, and the third is the classification pipeline that feeds the test set into the SVM classification component. Readers are capable of loading similarity or feature matrices and feeding them into visualizations or classifiers. The classification returns a new scalar field on the test data set, which can then be changed and adjusted by the user through the manual procedures explained in Section 5.2. The resulting classification can be saved as a copy of the distance matrix or as a copy of the data matrix that originated the classification, together with class for



(a) Class definition branch by branch.

(b) Manual classification using the browser.

Fig. 11. Visual support for manual definition of labels.

future use. The saved data matrix can then be loaded as a training set in subsequent classification steps.

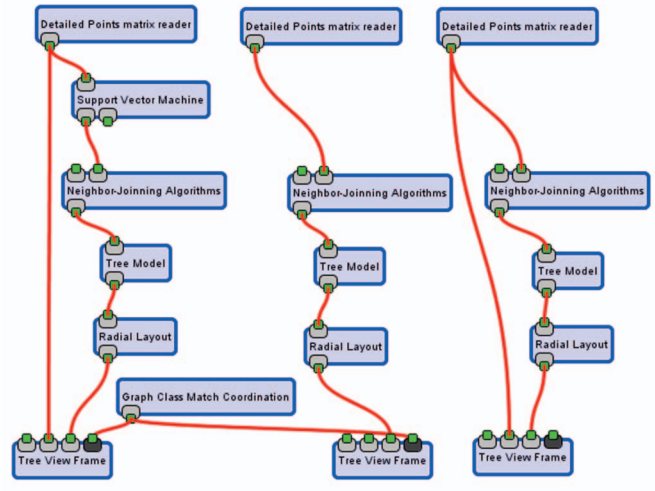
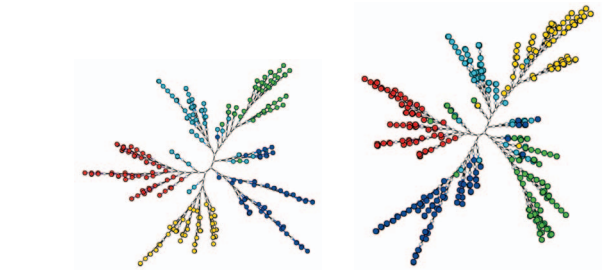



Fig. 12. Classification pipeline with visualization pipelines for test and training data sets.

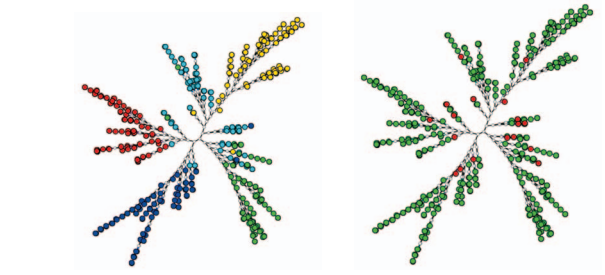
We tested that pipeline in various ways, but we illustrate progressive classification by extracting a data subset of the COREL data set containing 500 pictures and iteratively classifying it. We start with a labeled subset of 300 photos out of the 500 and add to the set 50 photos at each step. At each step, the classifier (SVM in this case) produces a result for the test set with added photos, which is then manually corrected by the user to increase the size of the classified set. This, in turn, can serve as a training set once new pictures are added. Figure 13 illustrates the process. In particular, Figure 13a shows the visualization of one training data set. The test data set colored by corresponding target classes is shown in Figure 13b. The data set after classification is shown in Figure 13c. It can be seen that the great majority of points were correctly classified.

When there is available information of target classes, that is, if the ground truth is known, it is possible to verify the classification results. Besides the usual values reflecting the number of correctly clas-



(a) Training data with 350 pictures.

(b) Test data with 500 pictures. Color is target class.



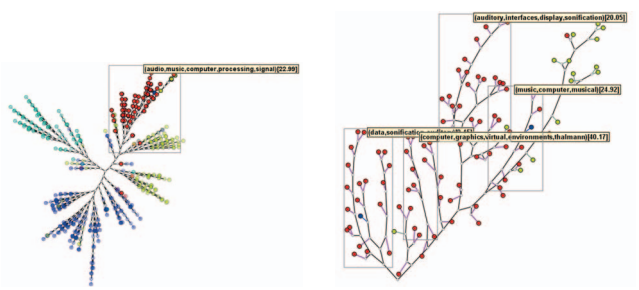
(c) Classified data with 500 pictures.

(d) Mismatch tree. Points in red were misclassified.

Fig. 13. Two steps of classification. Misclassified points: 25/500 (5%).

sified instances and other conventional measures of classifications, it is possible, using another supporting tool, to verify the difference between the correct classification and that obtained using also the trees, precisely locating points that were mistakenly classified. Figure 13d shows the results of this layout matching tool applied to the trees in Figures 13b and 13c. By interacting with that users may be inspired to find the reasons for classification failure in some cases. The *Graph Class Match Coordination* module, which realizes the matching of trees, can be seen in the pipeline of Figure 12.

Coordination is an integral part of the system. Selections can be coordinated amongst the various projections and trees. One advantage of coordinating trees with projections is to be able to identify, for a group well formed in the projection, a possible structure within the group in the tree with possible formation of subgroups. This can compensate for the poor local neighborhood reconstruction common to most projection techniques. Also, for groups that are not well formed in the projection, the coordination with trees may suggest hypotheses related to similarity calculation or to the feature extraction process.




(a) Global NJ tree. Points highlighted on top branch correspond to area highlighted in Figure 1.

(b) Detail of one of the branches from highlighted portion in 14a.

Fig. 14. NJ tree corresponding to same text data set as Figure 1.

The processes illustrated above can serve the purpose of supporting classification of many types of multidimensional data, since the visualizations take as starting point feature sets or similarity calculations. The system is prepared to handle most functions presented before for any data set. What differs is the handling of the manual classifier (the individual labeler) that has to be attached to a proper browser. Additional functions may, naturally, be needed for other data types. The system currently allows exploration and classification of text. Classification procedures are still rudimentary, but there is effective functionality for determination of topics in branches. Figure 14 shows an example for a text data set. Starting from the same LSP projection of Figure 1, after determining topics of various areas on that map, the group with predominantly red points is selected on the LSP and, by coordination, shown on the corresponding tree (Figure 14a). From that part of the tree, whose general content reflects computer audio, another branch is selected and highlighted in Figure 14b. One can see that, in the selected branch, different sub-branches are treating the subjects of computer music, auditory interfaces, data sonification and audio for virtual environments.

It is also possible to coordinate, by identity, visualizations of data types of different natures (such as image and text), provided the file identifiers for image and corresponding text are named equally. For instance, one can visualize and coordinate textual and image representations of protein sequences or of X-rays and their reports, and so on.

## 6 Conclusions and Future Work

This paper proposes solutions for processing time and visual overload problems of phylogenetic similarity trees for the purpose of multidimensional data visualization. The first problem was handled by employing faster Neighbor Joining implementations or Neighbor Joining strategy. Overload was relieved by a linear graph rewriting procedure based on node promotion.

Numerical and visual comparisons have shown that the FAST algorithm is actually the fastest with small loss in precision, and that the Rapid algorithm accelerates its original version resulting in the same structure, therefore the same precision as the original NJ tree. Use of visual space has gained considerable improvement, since an average of 51% of the virtual nodes are eliminated by promotion. These procedures should endorse larger employment of similarity trees as a precise and fast technique that supports analysis in both overall and detail tasks with the same properties. Trees are capable of examining the original space locally and globally with similar precision. The tree building and visualization processes need no parametrization from the user.

An innovative form, based on the trees, of a classical classification application was also presented. The set of tools for the application of visual mining of images is made possible by the properties offered by trees complemented by the possibilities offered by multidimensional projections.

Trees are built from similarity matrices or from vector spaces followed by similarity calculations, giving it flexibility to work with different types of information as input. While discussions on proper feature spaces or similarity functions are out of the scope here, we intend to employ the trees as a platform to support converging to proper definitions of vector space and similarity relationships. Additionally, our next steps include building trees from very large, disk-based data sets [41], to extend the use of similarity based visualizations to massive data sets. Another very promising venue is to use the matrix perturbation process presented in this paper as a means for feedback into the classification process. The system separate pipelines are to be integrated into a fully functional visual classification system.

## Acknowledgments

Authors wish to acknowledge CNPq (INCT - MACC 573710/2008-2 and 301295/2008-5), FAPESP and FAPEMIG Brazilian financial agencies. We are grateful to Fernando V. Paulovich and Rafael M. Martins for coding the core of VisPipeline.

## REFERENCES


[1] C. Bachmaier, U. Brandes, and B. Schlieper. Drawing Phylogenetic Trees. In *Algorithms and Computation*, volume 3827 of *Lecture Notes in Computer Science*, pages 1110–1121, 2005.

[2] B. B. Bederson, B. Shneiderman, and M. Wattenberg. Ordered and Quantum Treemaps: Making Effective Use of 2D Space to Display Hierarchies. *ACM Transaction on Graphics*, 21(4):833–854, October 2002.

[3] S. Büttcher, C. L. A. Clarke, and G. V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, Cambridge, MA, USA, 2010.

[4] M. Chalmers. A Linear Iteration Time Layout Algorithm for Visualising High-Dimensional Data. In *Proceedings of the 7th Conference on Visualization (VIS'96)*, pages 127–131, San Francisco, CA, USA, 1996.

[5] Y. Chen, L. Wang, M. Dong, and J. Hua. Exemplar-based Visualization of Large Document Corpus. *IEEE Transactions on Visualization and Computer Graphics*, 15:1161–1168, 2009.

[6] L. Cinque, S. Levialdi, A. Malizia, and K. Olsen. A Multidimensional Image Browser. *Journal of Visual Languages and Computing*, 9(1):103–117, 1998.

[7] C. Cortes and V. Vapnik. Support-Vector Networks. *Machine Learning*, 20(3):273–297, 1995.

[8] T. M. Cover and P. E. Hart. Nearest Neighbor Pattern Classification. *Institute of Electrical and Electronics Engineers Transactions on Information Theory*, 13:21–27, 1967.

[9] T. F. Cox and M. A. A. Cox. *Multidimensional Scaling*. Chapman & Hall/CRC, 2nd edition, 2000.

[10] A. M. Cuadros, F. V. Paulovich, R. Minghim, and G. P. Telles. Point Placement by Phylogenetic Trees and its Application for Visual Analysis of Document Collections. In *Proceedings of IEEE Symposium on Visual Analytics Science and Technology (VAST'2007)*, pages 99–106, Sacramento, CA, USA, 2007.

[11] J. Daniels, E. W. Anderson, L. G. Nonato, and C. T. Silva. Interactive Vector Field Feature Identification. *IEEE Transactions on Visualization and Computer Graphics*, 16:1560–1568, 2010.

[12] R. O. Duda and P. E. Hart. *Pattern Classification and Scene Analysis*. John Wiley & Sons Inc, New York, NY, USA, 1973.

[13] P. A. Eades. A Heuristic for Graph Drawing. *Congressus Numerantium*, 42:149–160, 1984.

[14] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer. *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.

[15] D. M. Eler, M. Y. Nakazaki, F. V. Paulovich, D. P. Santos, G. F. Andery, M. C. F. Oliveira, J. Batista-Neto, and R. Minghim. Visual Analysis of Image Collections. *The Visual Computer*, 25(10):923–937, 2009.

[16] I. Elias and J. Lagergren. Fast Neighbor Joining. In *Proceedings of the 32nd International Colloquium on Automata, Languages and Programming (ICALP'05)*, volume 3580, pages 1263–1274, 2005.

[17] J. Evans, L. Sheneman, and J. Foster. Relaxed Neighbor Joining: A Fast Distance-Based Phylogenetic Tree Construction Method. *Journal of Molecular Evolution*, 62(6):785–792, 2006.

[18] J. Fan, Y. Gao, and H. Luo. Hierarchical Classification for Automatic Image Annotation. In *Proceedings of the 30th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 111–118, New York, NY, USA, 2007.

[19] J. Fan, Y. Gao, and H. Luo. Integrating Concept Ontology and Multitask Learning to Achieve More Effective Classifier Training for Multilevel Image Annotation. *IEEE Transactions on Image Processing*, 17(3):407–426, march 2008.

[20] O. Gascuel and M. Steel. Neighbor-Joining Revealed. *Molecular Biology and Evolution*, 23(11):1997–2000, 2006.

[21] G. Griffin, A. Holub, and P. Perona. Caltech-256 Object Category Dataset. Technical Report 7694, California Institute of Technology, 2007.

[22] M. A. Hearst, S. T. Dumais, E. Osman, J. Platt, and B. Scholkopf. Support Vector Machines. *IEEE Intelligent Systems and their Applications*, 13(4):18–28, 1998.

[23] I. Jolliffe. *Principal Component Analysis*. Springer-Verlag, New York, NY, USA, 2nd edition, 2002.

[24] J. Li and J. Z. Wang. Automatic Linguistic Indexing of Pictures by a Statistical Modelin Approach. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25:1075–1088, 2003.

[25] T. Mailund, G. S. Brodal, R. Fagerberg, C. N. S. Pedersen, and D. Phillips. Recrafting the Neighbor-Joining Method. *BMC Bioinformatics*, 7(29), 2006.

[26] B. Moghaddam, Q. Tian, N. Lesh, C. Shen, and T. S. Huang. PDH: A Human-Centric Interface for Image Libraries. In *Proceedings of IEEE International Conference on Multimedia and Expo*, pages 901–904, New York, NY, USA, July 2002.

[27] G. Nguyen and M. Worring. Interactive Access to Large Image Collections using Similarity-Based Visualization. *Journal of Visual Languages & Computing*, 19(2):203–224, 2008.

[28] F. V. Paulovich, D. M. Eler, J. Poco, C. P. Botha, R. Minghim, and L. G. Nonato. Piecewise Laplacian-based Projection for Interactive Data Exploration and Organization. *IEEE Computer Graphics Forum, Proceedings Eurovis 2011*, 30(3):1091–1100, 2011.

[29] F. V. Paulovich and R. Minghim. HiPP: A Novel Hierarchical Point Placement Strategy and its Application to the Exploration of Document Collections. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1229–1236, 2008.

[30] F. V. Paulovich, L. G. Nonato, R. Minghim, and H. Levkowitz. Least Square Projection: A Fast High Precision Multidimensional Projection Technique and its Application to Document Mapping. *IEEE Transactions on Visualization and Computer Graphics*, 14(3):564–575, 2008.

[31] F. V. Paulovich, C. T. Silva, and L. G. Nonato. Two-Phase Mapping for Projecting Massive Data Sets. *IEEE Transactions on Visualization and Computer Graphics*, 16:1281–1290, 2010.

[32] C. Plaisant, J. Grosjean, and B. B. Bederson. SpaceTree: Supporting Exploration in Large Node Link Tree, Design Evolution and Empirical Evaluation. *IEEE Symposium on Information Visualization*, page 57, 2002.

[33] K. Rodden, W. Basalaj, D. Sinclair, and K. R. Wood. Evaluating a Visualization of Image Similarity as a Tool for Image Browsing. In *Proceedings of IEEE InfoVis*, pages 36–43, San Francisco, CA , USA, 1999.

[34] G. Rozenberg, editor. *Handbook of Graph Grammars and Computing by Graph Transformation*, volume 1. World Scientific Publishing Company, 1997.

[35] N. Saitou and M. Nei. The Neighbor-Joining Method: A New Method for Reconstructing Phylogenetic Trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.

[36] M. Simonsen, T. Mailund, and C. N. Pedersen. Rapid Neighbour-Joining. In *Proceedings of WABI 2008*, pages 113–122, Karlsruhe, Germany, September 2008.

[37] J. A. Studier and K. J. Kepler. A Note on the Neighbour-Joining Method of Saitou and Nei. *Molecular Biology and Evolution*, 5:729–731, 1988.

[38] E. Tejada, R. Minghim, and L. G. Nonato. On Improved Projection Techniques to Support Visual Exploration of Multidimensional Data Sets. *Information Visualization*, 2(4):218–231, 2003.

[39] Q. Tian, B. Moghaddam, and T. S. Huang. Visualization, Estimation and User-Modeling for Interactive Browsing of Image Libraries. In *ACM International Conference on Image and Video Retrieval*, pages 7–16, London, UK, 2002.

[40] T. J. Wheeler. Large-Scale Neighbor-Joining with NINJA. In *Proceedings of WABI 2009*, pages 375–389, Philadelphia, PA, USA, 2009.

[41] T. J. Wheeler. Large-Scale Neighbor-Joining with NINJA. In *Proceedings of the 9th International Conference on Algorithms in Bioinformatics*, pages 375–389, Philadelphia, PA, USA, 2009.

[42] M. Worring, O. de Rooij, and T. van Rijn. Browsing Visual Collections Using Graphs. In *Multimedia Information Retrieval*, pages 307–312, Augsburg, Germany, 2007.